```
# Exact certificate for one cycle (seed from argv), over Q(sqrt5), plus exact checks
of the table symmetry S from cycle 0.
import sys, numpy as np, io, contextlib, sympy as sp, json
from sympy.polys.matrices import DomainMatrix
seed=int(sys.argv[1])
ns={}
with contextlib.redirect_stdout(io.StringIO()):
exec(open('46_table_only_pairing.py').read().split("# (e)")[0],ns)
G,N,res,conjugator,analyse,pairing,A=[ns[k] for k in "G N res conjugator analyse
pairing A".split()]
B=[0]
for r in range(N):
    if np.linalg.matrix_rank(G[np.ix_(B+[r],B+[r])])>len(B): B.append(r)
    if len(B)==8: break
K=sp.QQ.algebraic_field(sp.sqrt(5)); phi=(1+sp.sqrt(5))/2
def esign(e):
    a,b=[sp.Rational(t) for t in (sp.Poly(K.to_sympy(e),sp.sqrt(5)).all_coeffs()
[::-1]+[0,0])[:2]]
    if b==0: return int(sp.sign(a))
    if a==0: return int(sp.sign(b))
    if sp.sign(a)==sp.sign(b): return int(sp.sign(a))
    return int(sp.sign(a)) if a*a>5*b*b else int(sp.sign(b))
GBq=sp.Matrix(G[np.ix_(B,B)].tolist()); Cq=(GBq.inv()*sp.Matrix(G[B,:].tolist())).T
D=lambda X: DomainMatrix.from_Matrix(X).convert_to(K); GBD,CD=D(GBq),D(Cq)
p=res[seed][1]
M=sp.Matrix([[Cq[p[B[b]],i] for b in range(8)] for i in range(8)]); x=sp.symbols('x')
cp=M.charpoly(x).as_expr(); fs=[f for f,_ in sp.factor_list(cp,extension=sp.sqrt(5))
[1]]
MD=D(M); P=[]
for f in fs:
    F=DomainMatrix.zeros((8,8),K)
    for c in sp.Poly(f,x).all_coeffs(): F=F*MD+DomainMatrix.eye(8,K)*K.from_sympy(c)
    Kn=F.nullspace().transpose(); P.append(Kn*
(Kn.transpose()*GBD*Kn).inv()*Kn.transpose()*GBD)
# identify which factor is sector A: the one containing the eigenvalue exp(2*pi*i/30)
(exact test: f(zeta)=0 over Q(zeta30))
# exact identification of sector A = channels {1,29,11,19}: its eigenvalue sum is
2cos12+2cos132 = 1/phi,
# so its factor's x^3 coefficient is -1/phi = (1-sqrt5)/2 (sector B gives +phi).
iA=[k for k,f in enumerate(fs) if sp.simplify(sp.Poly(f,x).coeff_monomial(x**3)-(1-
sp.sqrt(5))/2)==0][0]
order=[iA,1-iA]
V=[(CD*P[k].transpose()) for k in order]; VK=[[[V[t][r,j].element for j in range(8)]
for r in range(N)] for t in range(2)]
lev=[ (CD[r:r+1,:]*P[order[0]].transpose()*GBD*P[order[0]]*CD[r:r+1,:].transpose())
[0,0].element for r in range(N)]
```

```
Lv=list({K.to_sympy(l) for l in lev}); assert len(Lv)==2
lo,hi=(Lv[0],Lv[1]) if esign(K.from_sympy(sp.radsimp(Lv[1]-Lv[0])))>0 else
(Lv[1],Lv[0])
inner=[r for r in range(N) if K.to_sympy(lev[r])==lo]; outer=[r for r in range(N) if
K.to_sympy(lev[r])==hi]
def canon(r):
    v=VK[0][r]; k=next(i for i,e in enumerate(v) if e!=K.zero); a=v[k]
    return tuple(K.to_sympy(e/a) for e in v), esign(a)
dirs={}
for o in outer: dirs.setdefault(canon(o),[]).append(o)
cnt=[len(dirs.get(canon(i),[])) for i in inner]
part={i:dirs[canon(i)][0] for i in inner if len(dirs.get(canon(i),[]))==1}
f_=K.from_sympy(phi); fi=K.from_sympy(sp.radsimp(1/phi))
okA=len(part)==120 and all(all(VK[0][o][j]==f_*VK[0][i][j] for j in range(8)) for i,o
in part.items())
okB=len(part)==120 and all(all(VK[1][o][j]==-fi*VK[1][i][j] for j in range(8)) for
i,o in part.items())
# numerical predecessor
Pm,splits=analyse(p); numpart,*_=pairing(Pm,A)
out={"seed":seed,"levels":[str(lo),str(hi)],"counts":
[len(inner),len(outer)],"partners_per_inner":sorted(set(cnt)),
     "pairs":len(part),"outer_used_once":len(set(part.values()))==len(outer),
     "partner_relation_values":sorted({int(G[i,o]) for i,o in part.items()}),
     "sectorA_partner=phi*element":okA,"sectorB_partner=-(1/phi)*element":okB,
     "exact_pairing==numerical_pairing":{int(k):int(v) for k,v in part.items()}==
{int(k):int(v) for k,v in numpart.items()}}
if seed>0:
    s=conjugator(res[0][1],p); S=np.array([s[r] for r in range(N)])
    ns0={}
    base=json.load(open('51_cycle0_pairing.json'))
    part0={int(k):v for k,v in base.items()}
    out["S_permutation"]=sorted(S.tolist())==list(range(N))
    out["S_preserves_G_exactly(57600 integer
entries)"]=bool((G[np.ix_(S,S)]==G).all())
    out["S_conjugates_cycle0_to_cycle_k"]=bool((S[res[0][1]]==p[S]).all())
    out["S_transports_exact_pairing0_to_exact_pairing_k"]={int(S[i]):int(S[o]) for
i,o in part0.items()}=={int(k):int(v) for k,v in part.items()}
else:
    json.dump({int(k):int(v) for k,v in
part.items()},open('51_cycle0_pairing.json','w'))
print(json.dumps(out))
```